

miniFlink: Declarative Stream Processing Semantics in OCaml

MINIFLINK PROJECT, Independent, Russia

We present miniFlink, a functional implementation of Apache Flink semantics in OCaml that demonstrates how declarative stream processing reduces accidental complexity. The key insight is the KEYED type class: by encoding the grouping key once in the type, operators `window`, `enrich`, and `dedup` require no explicit `~key:` parameter, making the pipeline read as a specification rather than an implementation. We implement event time, watermarks, tumbling/sliding windows, stateful operators, exactly-once checkpointing, table and join semantics, retractions for late data, and production layers (dead letter queue, graceful shutdown, Prometheus metrics) in approximately 1200 lines across 42 files. A parallel backend supports both OCaml 4 (Thread + Mutex) and OCaml 5 (Domain + Atomic) with measured speedup of $2.48\times$ on 4 workers (1 CPU).

ACM Reference Format:

miniFlink Project. 2026. miniFlink: Declarative Stream Processing Semantics in OCaml. 1, 1 (May 2026), 6 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 DECLARATIVITY AS A FIRST PRINCIPLE

Traditional stream processing frameworks require the programmer to explicitly state how to group events, how to serialize them, and how to handle null values at every operator. Consider the same pipeline in Kafka Streams (Java) and miniFlink (OCaml):

Listing 1. Kafka Streams (Java) — imperative

```
KStream<String, Telemetry> stream =
    builder.stream("vehicles.telemetry");
KGroupedStream<String, Telemetry> grouped = stream
    .filter((k, v) -> v != null)
    .groupBy((k, v) -> v.getDeviceId(),
        Grouped.with(Serdes.String(), telemetrySerde));
KTable<Windowed<String>, Stats> stats = grouped
    .windowedBy(TimeWindows.ofSizeWithNoGrace(
        Duration.ofSeconds(30)))
    .aggregate(Stats::empty,
        (k, v, acc) -> acc.add(v),
        Materialized.as("stats-store"));
stats.toStream()
    .flatMapValues(Rules::check)
    .filter((k, v) -> v != null)
    .to("fleet.alerts");
```

Author's address: miniFlink Project, Independent, Russia.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/5-ART

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Listing 2. miniFlink (OCaml) — declarative

```
let pipeline source =
  source
  |> Event.with_watermarks ~latency:(seconds 3)
  |> Pipe.enrich (module Telemetry)
    ~from:devices
    ~merge:(fun t dev -> { t with device = dev })
  |> Pipe.window (module Telemetry) (tumbling (seconds 30))
  |> Pipe.aggregate Rules.compute
  |> Pipe.flat_map (Rules.check Rules.fleet)
  |> Pipe.dedup (module Alert)
    ~rule:(fun a -> a.rule)
    ~cooldown:(minutes 5)
```

The differences are structural, not cosmetic:

- **No key_by.** The KEYED type class encodes the grouping key in the type once; `window`, `enrich`, and `dedup` extract it automatically.
- **No type annotations** inside the `|>` chain. Hindley-Milner inference propagates types through all operators.
- **No null checks.** `Option` makes absence explicit; `None` cannot be dereferenced.
- **Format in one place.** Changing JSON to Protobuf means replacing one `Codec` value; the pipeline is untouched.

2 ARCHITECTURE

miniFlink is organized into four orthogonal layers. Each layer depends only on layers below it; the core never imports reliability or parallelism modules.

Table 1. Layer organization

Layer	Modules	Responsibility
Core	stream, event, pipe, keyed, table, codec	Immutable. No external dependencies
Domain	domain, rules	[[@deriving yojson]]. Pure functions
Parallelism	channel_v4/v5, parallel_v4/v5	OCaml 4: Thread+Mutex. OCaml 5
Reliability	dlq, shutdown, metrics, barrier, schema	Each: .mli + _noop + _default.

The Makefile detects OCAML_MAJOR and copies the appropriate `channel_v{4,5}.ml` and `parallel_v{4,5}.ml` before compilation.

3 CORE ABSTRACTIONS

3.1 Pull Stream

```
type 'a t = unit -> 'a option
```

A stream is a function. The call stack *is* the operator graph. No callbacks, no threads, no allocation per event beyond the value itself. `map`, `filter`, `flat_map` compose without intermediate buffers.

3.2 Event

```

type 'a t =
  | Data      of 'a * timestamp    (* value + event time *)
  | Watermark of timestamp        (* lower bound on future ts *)
  | Retract   of 'a * timestamp    (* revocation of prior Data *)

```

All three constructors flow through the same stream. Operators match exhaustively — adding a constructor forces all operators to handle it at compile time.

3.3 KEYED Type Class

```

module type S = sig
  type t
  val key : t -> string
end

(* Registered once per domain type *)
module Telemetry = Keyed.Make(struct
  type t = telemetry
  let key t = t.device_id
end)

```

Operators receive (module Telemetry) as a first-class module and call `K.key` internally. The user never writes `~key:(fun t -> t.device_id)` again.

3.4 Window

The locally abstract type annotation breaks the occurs-check that would otherwise create the recursive type $\alpha = \text{string} \times \alpha \text{ list}$:

```

let window (type a) ?(latency=0)
  ~(key : a -> string) spec
  (upstream : a Event.t Stream.t)
  : (string * a list) Event.t Stream.t = ...

```

Window state is a `WMap.t` from (key, start, stop) triples to event lists. Watermarks trigger partition of open windows into *closed* (fired) and *open* sets; closed windows emit their accumulated events downstream.

4 EVENT TIME AND WATERMARKS

Each event carries its origin timestamp independently of processing time. `with_watermarks` inserts `Watermark(max_seen - latency)` after each new maximum timestamp. The invariant:

$$\forall t < \text{wm} : \text{no future Data}(_, t) \text{ will arrive}$$

This allows deterministic window closure without wall-clock polling. Watermarks are monotone by construction — `max_seen` is a `ref` that only grows.

5 EXACTLY-ONCE CHECKPOINTING

A checkpoint is a triple (*epoch, offset, state*) stored in `Checkpoint.Store`. The runner:

- (1) Reads N events from source.
- (2) On failure: restores state from latest checkpoint, calls `source.seek(offset)`, resumes.
- (3) On success every K events: atomically saves ($epoch+1$, `get_offset()`, `get_state()`).

The key invariant is *ordered commit*: flush sink before writing checkpoint; write checkpoint before committing source offset. Violated ordering creates either duplicates (checkpoint before flush) or gaps (offset before checkpoint).

6 TABLE AND JOIN

```
type ('k,'v) table = 'k -> 'v option

let of_list pairs = Hashtbl.find_opt (Hashtbl.of_seq ...)
let of_stream ~key src = fun k -> pump src; Hashtbl.find_opt h k
```

A table is a function. Snapshot tables call `pump` on each lookup to drain pending updates from an upstream stream. `lookup_join` is a one-liner:

```
let lookup_join (module K) ~from ~merge upstream =
  emap (fun v -> merge v (from (K.key v))) upstream
```

`interval_join` buffers both streams; events match when keys are equal and $|t_A - t_B| \leq \text{window_ms}$. The combined watermark is $\min(\text{wm}_A, \text{wm}_B)$.

7 RETRACTIONS FOR LATE DATA

When a late Data arrives after a window has fired, the operator emits:

$$\text{Retract}(v_{\text{old}}, t) \rightarrow \text{Data}(v_{\text{new}}, t)$$

Two aggregator strategies:

INVERTIBLE `remove(acc, x)` is defined. Late data: $O(1)$ update via `add/remove`. Examples: sum, count, average.

RECOMPUTE No inverse. Late data triggers full recomputation over the buffered event list: $O(n)$. Examples: max, min, percentile.

Both strategies produce identical external behaviour: the downstream sees a `Retract/Data` pair and can perform an upsert.

8 ORTHOGONAL RUNTIME

Each production capability has three files: `foo.mli` (contract), `foo_noop.ml` (zero-overhead stub), `foo_default.ml` (production implementation). `runtime.ml` assembles them by mode:

```
type mode = Noop | Log | Parallel | Prod

(* Noop: zero overhead, for tests *)
(* Log:  stderr metrics + SIGTERM handler *)
(* Prod: Prometheus HTTP :9090 + shutdown + DLQ log *)
```

Dead Letter Queue. `safe_decode` wraps every codec call; decode errors send the raw bytes to the DLQ with topic, error message, and timestamp instead of crashing the pipeline.

Graceful Shutdown. A `guarded_source` checks `is_requested()` before each poll. On SIGTERM: source returns `None`, pipeline drains its buffer, sink flushes, process exits.

Prometheus Metrics. `metrics_log` implements counter, gauge, and histogram (7 buckets, $1\mu s-1s$) with thread-safe `Mutex`-protected state and a TCP HTTP server that serves the exposition text format on `GET /metrics`.

9 PARALLELISM

Data parallelism via key sharding: `hash(key) mod N` routes each event to a dedicated worker. Workers share no state — window correctness follows from the invariant that all events for a given key go to the same shard.

Table 2. Benchmark: 1M events, 1000 devices, tumbling 30s, OCaml 4, 1 CPU

Mode	ev/s	Time (s)	Speedup
Sequential	209,293	4.78	1.00×
1 worker	151,570	6.60	0.72×
2 workers	405,775	2.46	1.94×
4 workers	519,463	1.93	2.48×
8 workers	599,988	1.67	2.87×

The sub-linear speedup on 1 CPU is expected: OCaml 4’s GIL prevents true parallelism, but channel-mediated pipelining hides latency. On OCaml 5 with `Domain`, channel overhead drops from ~ 106 ns (`Mutex`) to ~ 30 ns (`Atomic`), and real 4-core parallelism is expected to yield 3.5–3.8×

10 TESTING

Harness. A deterministic test context `(’a, ’b) t` feeds events via `push/push_wm` and collects output via `run`. No broker, no threads, no wall clock.

Unit tests (22): stream laws (map identity, filter composition, `flat_map` count), watermark monotonicity, window correctness, dedup cooldown, enrich left-join semantics, rule thresholds, full pipeline integration.

Property tests (7×100 –500 cases via `QCheck`):

- (1) $\sum_w \text{count}(w) = |\text{input}|$ — window preserves all events.
- (2) `dedup(dedup(s)) = dedup(s)` — dedup is idempotent.
- (3) Watermarks are monotone for any event order.
- (4) `enrich` preserves event count (left join).
- (5) `critical` $\Leftrightarrow$ `speed` $> 1.3 \times \text{limit}$ — for all `float` pairs.
- (6) `map` preserves event count.
- (7) $|\text{filter}(s)| \leq |s|$.

11 CONCLUSION

miniFlink demonstrates that the core semantics of Apache Flink — event time, watermarks, windowing, stateful operators, exactly-once, table/join, retractions, and production reliability layers — fit in ~ 1200 lines of OCaml. Two design decisions proved most consequential:

- (1) The `KEYED` type class eliminates repeated key extraction, making the pipeline read as a specification.

(2) The `(type a)` locally abstract type annotation in `Pipe.window` avoids the recursive type $\alpha = \text{string} \times \alpha \text{ list}$ that otherwise requires `Obj.magic`.

The OCaml module system's ability to accept modules as runtime values (first-class modules) is what makes both design decisions expressible without metaprogramming or code generation.